

NAME: S.GAYATHERI

REG NO:192521306

COURSE CODE: CSA0619

COURSE NAME : Design

Analysis of Algorithm

CO4 – AT3 --Comparative Analysis

Q12. Knapsack Problem Comparison: 0/1 DP vs Fractional Greedy

Given Data

Item	Weight	Value	Value/Weight Ratio
Item 1	10	60	6
Item 2	20	100	5
Item 3	30	120	4

Knapsack Capacity = 50

1. 0/1 Knapsack Using Dynamic Programming

In the 0/1 Knapsack Problem, an item can either be:

- Selected completely (1), or
- Not selected (0)

Items cannot be divided.

Step 1: Possible Selections

Option 1: Item 1 + Item 2

- Total Weight = $10 + 20 = 30$
- Total Value = $60 + 100 = 160$

Option 2: Item 1 + Item 3

- Total Weight = $10 + 30 = 40$
- Total Value = $60 + 120 = 180$

Option 3: Item 2 + Item 3

- Total Weight = $20 + 30 = 50$
- Total Value = $100 + 120 = 220$

Option 4: All Items

- Total Weight = $10 + 20 + 30 = 60$

- Capacity = 50
- Therefore, **not possible**

Optimal 0/1 Knapsack Solution

Selected Items: Item 2 and Item 3

Maximum Value = 220

Total Weight = 50

2. Fractional Knapsack Using Greedy Method

In the Fractional Knapsack Problem, items can be divided into fractions.

The greedy strategy selects items according to the highest:

$$\text{Value-to-Weight Ratio} = \frac{\text{Value}}{\text{Weight}}$$

Step 1: Sort by Value/Weight Ratio

Item	Value/Weight Ratio
Item 1	6
Item 2	5
Item 3	4

Selection order:

<i>Item 1 → Item 2 → Item 3</i>

Step 2: Select Items

Select Item 1

- Weight = 10
- Value = 60
- Remaining Capacity =

$$50 - 10 = 40$$

Select Item 2

- Weight = 20

- Value = 100
- Remaining Capacity =

$$40 - 20 = 20$$

Select Fraction of Item 3

Item 3 weighs 30, but only 20 capacity remains.

Fraction selected:

$$\frac{20}{30} = \frac{2}{3}$$

Value obtained:

$$\frac{2}{3} \times 120 = 80$$

Total Fractional Knapsack Value

$$60 + 100 + 80 = \boxed{240}$$

Thus:

$$\boxed{\text{Maximum Value} = 240}$$

$$\boxed{\text{Selection: Item 1} + \text{Item 2} + \frac{2}{3} \text{ of Item 3}}$$

3. Comparison of Results

Comparison Parameter	0/1 Knapsack (DP)	Fractional Knapsack (Greedy)
Item Selection	Complete item only	Complete or partial item
Method	Dynamic Programming	Greedy Algorithm
Selection Strategy	Evaluates combinations	Selects highest value/weight ratio
Items Selected	Item 2 and Item 3	Item 1, Item 2, and 2/3 of Item 3
Total Weight	50	50

Maximum Value	220	240
Optimal Solution	Yes	Yes
Time Complexity	$O(nW)$	$O(n \log n)$
Space Complexity	$O(nW)$	$O(n)$ or $O(1)$ extra space after sorting

4. Why Does Greedy Work for Fractional Knapsack?

Greedy selects the item with the highest value per unit weight first.

For example:

- Item 1 gives value 6 per unit weight
- Item 2 gives value 5 per unit weight
- Item 3 gives value 4 per unit weight

Since fractional items can be divided, the remaining capacity can always be filled with a portion of the next best item.

Therefore, choosing the highest value-to-weight ratio first always maximizes the total value.

Example

After selecting Items 1 and 2:

- Remaining capacity = 20
- Instead of rejecting Item 3 completely, the algorithm takes 2/3 of it.

This flexibility makes the greedy choice optimal.

5. Why Does Greedy Not Always Work for 0/1 Knapsack?

In 0/1 Knapsack, items cannot be divided.

A greedy algorithm may choose an item with the highest value/weight ratio, but that decision may prevent a better combination of items from fitting into the knapsack.

Example from the Given Problem

Greedy order based on ratio:

Item 1 → Item 2 → Item 3

Greedy 0/1 selection:

- Select Item 1 $\rightarrow$ Weight = 10, Value = 60
- Select Item 2 $\rightarrow$ Weight = 20, Value = 100
- Remaining capacity = 20
- Item 3 cannot fit

Total value:

$$60 + 100 = \boxed{160}$$

However, the optimal 0/1 solution is:

- Item 2 + Item 3
- Weight = 50
- Value =

$$100 + 120 = \boxed{220}$$

Therefore:

$$\boxed{\text{Greedy solution } 160 < \text{Optimal DP solution } 220}$$

This clearly shows why a greedy strategy is not guaranteed to be optimal for the 0/1 Knapsack Problem.

6. Computational Complexity

0/1 Knapsack Dynamic Programming

Let:

- n = number of items
- W = knapsack capacity

Time Complexity

$$\boxed{O(nW)}$$

The DP algorithm examines every item for every possible capacity.

Space Complexity

Using a standard DP table:

$$O(nW)$$

Fractional Knapsack Greedy Algorithm

The major operation is sorting items according to their value-to-weight ratio.

Time Complexity

$$O(n \log n)$$

Sorting dominates the computation.

Space Complexity

$$O(n)$$

This may be lower depending on the sorting implementation.

7. Memory Usage in Dynamic Programming

The standard DP method creates a table:

$$(n + 1) \times (W + 1)$$

For example, if:

- Number of items = 3
- Capacity = 50

The DP table contains:

$$4 \times 51 = 204$$

states.

For large capacity values, memory consumption becomes significant.

Memory Optimization

Instead of storing the complete two-dimensional table, a one-dimensional array can be used.

This reduces memory from:

$$O(nW)$$

to:

$$\boxed{O(W)}$$

However, the capacity must be processed in reverse order to ensure that each item is selected only once.

8. Critical Evaluation

0/1 Dynamic Programming

Strengths

- Guarantees an optimal solution.
- Suitable when items cannot be divided.
- Examines different combinations systematically.
- Useful for discrete resource allocation problems.

Weaknesses

- Higher computational cost for large capacity values.
- Requires significant memory with a 2D DP table.
- Can become inefficient when W is very large.

Fractional Greedy

Strengths

- Faster and simpler.
- Requires less memory.
- Guarantees an optimal solution for fractional items.
- Suitable for large-scale divisible resource allocation.

Weaknesses

- Cannot be used when resources are indivisible.
- Value/weight ratio alone may lead to a non-optimal solution in 0/1 cases.

- Does not evaluate all possible combinations.

9. Insights for Resource Allocation

The choice of algorithm depends mainly on whether resources are **divisible or indivisible**.

Use Fractional Knapsack (Greedy) When:

- Resources can be divided.
- Partial allocation is allowed.
- Fast execution is important.
- Examples include bandwidth allocation, investment allocation, and divisible raw materials.

Use 0/1 Knapsack (DP) When:

- Resources are indivisible.
- Each item must be fully accepted or rejected.
- An optimal solution is required.
- Examples include project selection, equipment allocation, task selection, and budget planning.

10. Final Conclusion

For the given problem:

0/1 DP Maximum Value = 220

Fractional Greedy Maximum Value = 240

The fractional knapsack produces a higher value because it allows partial selection of Item 3.

Dynamic Programming is the correct approach for **indivisible resources** because it guarantees the optimal combination. The Greedy approach is more computationally efficient but guarantees optimality only when items can be divided.

Final Insight

Divisible Resources $\Rightarrow$ Greedy Fractional Knapsack
--

Indivisible Resources $\Rightarrow$ Dynamic Programming 0/1 Knapsack
--

This comparison shows that the **nature of the resource allocation problem**, rather than only algorithm speed, should determine the most appropriate approach.

